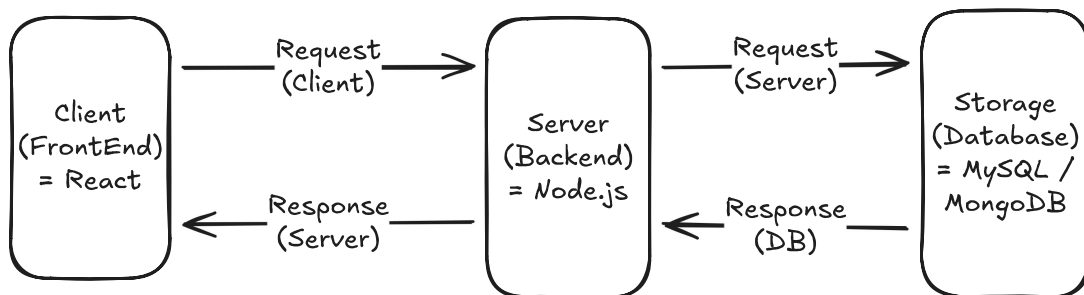


Topic: Node.js Introduction
Date: 11 January 2026

```
db.books.aggregate([
  {
    $lookup: {
      from: 'authors',
      localField: 'author_id',
      foreignField: '_id',
      as: 'author_details'
    }
  },
  {
    $lookup: {
      from: 'reviews',
      localField: '_id',
      foreignField: 'book_id',
      as: 'reviews'
    }
  }
])
```



Client - Frontend Application

1. Very specific to the browser
2. limited access to the system resources
3. Logic can be visible in the browser

Server - Backend Application

1. Business logic is kept here
2. can access the system resources
3. is running outside of the browser environment or has its own runtime environment

Python - Best for data modelling and analytics - AI

Java - Known for its security and OOPS concepts

Node.JS - faster execution and async

PHP - security and faster execution

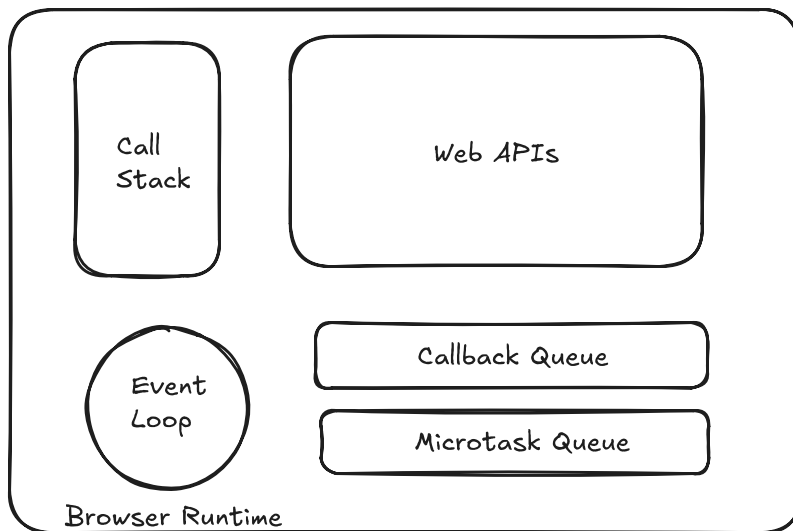
Node.js

It is an open source, cross platform Javascript runtime, which executes JS outside of browser.

- > Node.js is not a programming language
- > it is not a framework
- > it does not use multi threading
- **-> It is runtime environment

How Node.js works?

1. V8 Engine - it is built on V8 engine which was used in Chrome Browsers only
 - it converts JS code into machine code
2. libuv - handle asynchronous operations
3. Event Loop - manages the execution of the JS sync or async operations



Traditional Server

Request -> Waits -> Responds

Node.js

Request -> Delegates -> Continue -> Callback -> Response

- > concurrent requests can be entertained
- > high throughput with low memory usage.

Parallel and Concurrent

Parallel execution - it is executing multiple requests or processes at a single point of time

Task 1 and 2 are being executed / in-progress simultaneously at minute 0,

Task 1 and 3 are executed / in-progress simultaneously at minute 1

Task1
Task2
Task3



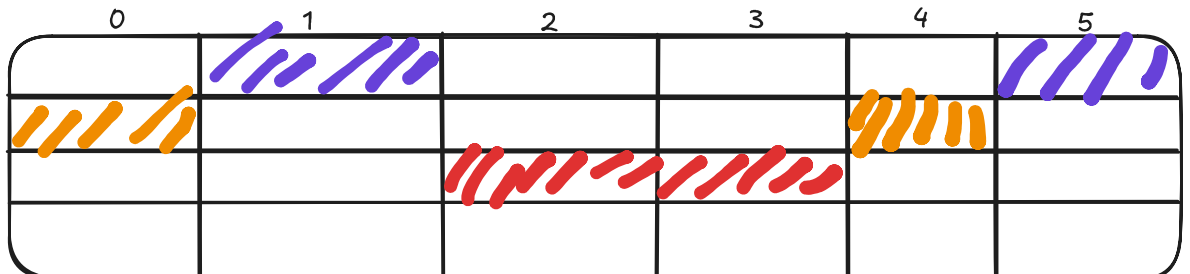


No tasks are being executed or in-progress simultaneously

Wait for the previous task to execute and then start

Concurrent Execution

The task is executed as soon as the processor is available.



Task 1 (Purple), Task 2(yellow), Task 3(Red)

Arrival of tasks - Task1 - 0, Task2 - 0, Task3 - 2

Task2 is in progress from 0 to 4

Task 1 is in progress from 1 to 5

Task 3 is in progress from 2 to 3

Task 1 and task 2 are in progress from 1 to 4

Task 1, task2 and task3 are in progress from 2 to 3.

Parallel is about doing many things at a time

Concurrency is about dealing with multiple things at a time

When to use Node.js

- > APIs
- > Real Time application
- > Streaming services
- > Microservices

package.json

- > contains the project metadata
- > acts as a dependency manager
- > scripts runner
- > configuration file

without package.json file npm does not know how the project works

name and version -> used by npm registry
follow the semantic versioning

1.0.0 -> major. minor. patch

^ -> means if any latest version is found install that for me

main - it defines the starting / entry point of the application.

scripts - it eliminates the need of long terminal commands

dependencies - it is the collection of the packages required by the application to run.

dev-dependencies - it is the collection of packages required by the application to be built

package-lock.json

it guarantee:

- > same dependency versions

-> same dependency tree

if lock file is not present different versions are installed
and with lock file exact matching versions are installed

node_modules

it is a directory containing

- > direct dependency
- > their dependencies
- > the dependencies of the dependencies

Whenever the node requires any module

- > checks within the same folder
- > checks in the node_modules
- > check within the parent folder
- > Global modules

Use gitignore file to ignore the node_modules into the repository.